

CAPSTONE ASSIGNMENT 2

Selenium Python Automation Framework

Login and Product Search Automation for AutomationExercise

Student Enrollment No.	12023002001058
Project / Assignment	Capstone Assignment 2
Target Application	https://automationexercise.com/
Primary Automation Tool	Selenium WebDriver with Python
Testing Frameworks	PyTest and unittest
Design Pattern	Page Object Model (POM)
Submission Date	24 September 2026

Project status: complete Assignment 2 baseline with 8/8 runtime tests passing in the recorded local execution.

Prepared for academic capstone submission

1. Executive Summary

This project implements a reusable Selenium WebDriver automation framework in Python for the AutomationExercise public e-commerce demonstration application. The framework is organized using the Page Object Model and separates test intent from browser mechanics, configuration, test data, reporting, and failure handling.

The implementation addresses the specified Assignment 2 requirements: Selenium WebDriver, PyTest, unittest, Page Object Model, utility classes, configuration management, CSV-based test data, screenshots on failure, and HTML reporting. The automated business scope covers login navigation and validation plus data-driven product search.

Recorded runtime result: 8 tests passed, 0 failed, 0 skipped. The execution also generated a self-contained HTML report. The recorded run used Windows 10, Python 3.11.4, PyTest 9.1.1 and completed in 252.53 seconds.

2. Assignment Requirements and Scope

The assignment specification requires a scalable Selenium Python framework using the following components:

Requirement	Implementation in this project
Selenium WebDriver	DriverFactory creates and configures the selected browser.
PyTest	Primary test execution layer and fixture infrastructure.
unittest	Independent unittest smoke test class for login-page contract validation.
Page Object Model	BasePage, HomePage, LoginPage and ProductsPage encapsulate UI behavior.
Utility Classes	Configuration, CSV reader, driver factory, logger and screenshot utilities.
Configuration Management	config/config.ini with environment-variable overrides and CLI browser/headless/base URL options.
CSV Test Data	data/test_data.csv drives product-search cases and invalid-login data.
Screenshots on Failure	PyTest hook captures screenshots for fixture-based failures.
HTML Reporting	pytest-html produces reports/html/report.html.

The target application selected for the project is AutomationExercise, as permitted by the assignment specification.

3. Framework Architecture

The project follows a layered architecture. Tests express business behavior, page objects own selectors and user actions, and utilities provide reusable framework services.

TEST LAYER	PAGE OBJECT LAYER	UTILITY / INFRASTRUCTURE
PyTest tests unittest test class assertions and scenarios	BasePage HomePage LoginPage ProductsPage	DriverFactory ConfigManager CSVReader Screenshot / Logger PyTest fixtures

Layer	Responsibility
Tests	Describe what must be true. They avoid raw WebDriver selectors.

Layer	Responsibility
Page Objects	Describe how the application is operated. They contain selectors and actions such as login and search.
BasePage	Provides common waiting, clicking, typing, text and URL operations.
Utilities	Handle browser creation, configuration, CSV reading, logging and screenshots.
conftest.py	Provides reusable PyTest fixtures, command-line overrides and failure screenshot integration.
Judge	Acts as a static quality gate for required structure and Python syntax.

4. Project Structure

The final project package contains the following main structure:

```
Capstone_Assignment_2_AutomationExercise_v1.2/ |-- config/ | `-- config.ini |-- data/ | `--
test_data.csv |-- docs/ | |-- ARCHITECTURE.md | |-- FEATURE_BACKLOG.md | |-- JUDGING.md | `--
TROUBLESHOOTING.md |-- judge/ | `-- judge.py |-- pages/ | |-- base_page.py | |-- home_page.py | |--
login_page.py | `-- products_page.py |-- tests/ | |-- test_login_pytest.py | |-- test_login_unittest.py
| `-- test_product_search_pytest.py |-- utils/ | |-- config_manager.py | |-- csv_reader.py | |--
driver_factory.py | |-- logger.py | `-- screenshot.py |-- conftest.py |-- pytest.ini |--
requirements.txt |-- run_tests.py |-- test_data_example.env `-- PROJECT_REPORT.pdf
```

The separation is intentional. A future feature such as cart automation can add a new page object and test layer without duplicating browser setup or reporting infrastructure.

5. Key Implementation Details

5.1 Browser Factory and Configuration

DriverFactory creates Chrome, Firefox or Edge drivers based on configuration. Headless execution can be enabled through configuration or the PyTest command line. Selenium Manager is used, so the project normally does not require a manually downloaded driver executable.

5.2 Page Object Model

The page classes encapsulate selectors and actions. For example, LoginPage owns the email field, password field, login button, login-error message, successful-login state and logout link. ProductsPage owns the product search controls and result extraction.

5.3 Synchronization and Robust Click Handling

BasePage uses explicit waits and keeps implicit waiting at zero. During runtime validation, the public site displayed full-viewport advertisement iframes that could intercept clicks. The framework therefore includes a narrowly scoped advertisement-overlay mitigation followed by a normal retry and a DOM-click fallback only if the WebDriver click remains intercepted.

5.4 Test Data

The CSV file contains three product-search cases (top, dress and jean), expected result headings and product fragments, plus invalid-login data. The tests consume this data instead of embedding test values in the test functions.

5.5 Reporting and Failure Evidence

PyTest is configured with pytest-html and writes a self-contained HTML report to reports/html/report.html. The failure hook captures screenshots into reports/screenshots/ when a WebDriver-based PyTest test fails.

6. Test Cases

ID	Test case	Expected outcome	Observed
TC01	Login page is reachable	Home page loads; login navigation and login page contract are valid.	PASS
TC02	Invalid credentials	Invalid-login error is displayed.	PASS
TC03	Valid credentials	Successful login is shown and logout returns to login page.	PASS
TC04	unittest login contract	Home/login page contract is valid using unittest.	PASS
TC05	Search: top	Search results heading and related result fragment are present.	PASS

ID	Test case	Expected outcome	Observed
TC06	Search: dress	Search results heading and related result fragment are present.	PASS
TC07	Search: jean	Search results heading and related result fragment are present.	PASS
TC08	CSV reader contract	Required CSV fields are present.	PASS

The positive-login test uses externalized credentials. No real password is stored in the project package or source control. In the recorded execution, the account credentials were supplied through the local environment.

7. Validation and Quality Evidence

7.1 Static Framework Judge

Feature	Points	Result
Project structure and separation of concerns	10	PASS
Selenium WebDriver / browser factory	10	PASS
Page Object Model	15	PASS
Configuration management	10	PASS
CSV test-data handling	10	PASS
PyTest login/search coverage	15	PASS
unittest coverage	10	PASS
Failure screenshots	10	PASS
HTML reporting	5	PASS
Maintainability / documentation	5	PASS
Total	100	100/100

The static judge also passed its Python syntax gate. The judge is intentionally treated as structural evidence rather than a substitute for real browser execution.

7.2 Recorded Runtime Execution

Metric	Observed value
Tests collected	8
Passed	8
Failed	0
Skipped	0
Execution time	252.53 seconds
HTML report	Generated at reports/html/report.html
Observed Python	3.11.4
Observed PyTest	9.1.1
Platform	win32 / Windows 10

This runtime result was captured from the local execution used for submission validation.

8. Runtime Defects Found and Corrected

The framework was tested incrementally rather than accepting a static score as proof of correctness. Two runtime issues were found and corrected:

Issue	Cause	Correction
Home page false negative	The initial page-load check assumed an unstable / non-existent H1 condition.	Changed the home-page contract to use the live document title plus a stable login navigation control.
Search false negatives for top and dress	The initial test required every returned product name to contain the literal search term, which did not match the live site behavior.	Changed the oracle to require the results heading, a non-empty result set and at least one result matching the data-driven expected product fragment.
Login click interception	A full-viewport third-party advertisement iframe could cover the Login button.	Added targeted ad-overlay mitigation and retry logic in BasePage.click(), with a final DOM-click fallback.

These changes improved the reliability of the test oracle and browser interaction without altering unrelated framework layers.

9. Requirement Traceability

Assignment requirement	Evidence in project
Selenium WebDriver	utils/driver_factory.py and WebDriver-based page objects
PyTest	tests/*.py, conftest.py, pytest.ini
unittest	tests/test_login_unittest.py
POM	pages/base_page.py, home_page.py, login_page.py, products_page.py
Utility Classes	utils/config_manager.py, csv_reader.py, driver_factory.py, logger.py, screenshot.py
Configuration Management	config/config.ini plus environment / CLI overrides
CSV Test Data	data/test_data.csv
Screenshots on Failure	conftest.py and utils/screenshot.py
HTML Reporting	pytest.ini and pytest-html configuration; report generated during runtime

10. Execution Instructions

From the project root, install dependencies and activate the virtual environment:

```
python -m venv .venv .\venv\Scripts\Activate.ps1 python -m pip install -r requirements.txt
```

Configure valid-login credentials only through environment variables or a local ignored configuration file:

```
$env:TEST_EMAIL="your_test_email@example.com" $env:TEST_PASSWORD="your_test_password"
```

Run the complete suite:

```
python run_tests.py
```

Run the static quality judge:

```
python judge/judge.py
```

11. Conclusion

The project delivers the requested Assignment 2 Selenium Python automation framework for AutomationExercise using a maintainable Page Object Model architecture. The framework separates browser infrastructure, configuration, data management, page behavior, tests and reporting so that the suite can be extended without

duplicating core setup logic.

The final baseline was validated both structurally and through live browser execution. The static judge reached 100/100 and the recorded end-to-end runtime execution completed with 8 passed, 0 failed and 0 skipped tests. The project is therefore ready for source-control submission and subsequent feature-by-feature review.

Appendix A - Submission Contents

Item	Purpose
PROJECT_REPORT.pdf	Formal project report for submission.
README.md	Setup, execution and framework overview.
pages/	Page Object Model implementation.
utils/	Reusable framework utilities.
tests/	PyTest and unittest automation scenarios.
config/	Application and execution configuration.
data/	CSV test data used by the automation suite.
docs/	Architecture, feature backlog, judging and troubleshooting documentation.
judge/	Static quality and structure judge.

Student Enrollment No.: 12023002001058

End of report.